L 简单题（定位：签到题）

给定一个 $n \times n$ 的从 1 到 $n \times n$ 依次排列的矩阵，要从里面每行每列恰好各删两个数字，问被删除数字的和最小是多少。

$n \le 10^3$

题解

实际上怎么删答案都一样，我们把每一个数字的贡献，拆成行标 $(x-1)\times n$ 加上列标 y 。

那么无论怎么删，行标和列标的贡献之和是确定的，直接输出解即可。

答案会超过 $\mathrm{int_max}$ ，如果忘了开 $\mathrm{long\ long}$ 会 wa 一发。

K 福利（定位：签到题）

一共两种选项，选了 A 的人一起平分 x 元，选了 B 的人各自单独拿 y 元。

有 n 个无私的人和 m 个自私的人。无私的人商量好一起决策，目标是让所有人拿到的钱总数最多。

自私的人会在无私的人决策完后决策，他们都会假设其他自私的人选选项 B 。

问最终一共拿走了多少钱。

$0 \le n, m, x, y \le 10^9$

题解

出题人云顶玩多了出出来的签到题。一共就两种大情况，但是要讨论好多小情况。

大情况1：所有无私的人都选选项 B。 这个情况比较好处理。

大情况2：部分无私的人选选项 A，逼自私的人选选项 B ，这个容易写挂，主要是 n, m, x, y 这四个变量中的某一些等于 0 的时候，非常容易挂，我作为验题人挂了一发，应该是挂的最少的人。

所以后面就加了很多样例，都是验题人挂的样例，最初的版本是 3 个非常弱的样例，最后的版本是 13 个样例，但因为时间来不及了，正赛的版本是 9 个样例。

赢（定位：签到题）

给你一个小写字符串 s，你可以插入最多 k 个字符，问插入完后，在这个字符串中最多能找到多少个字符串 lose。

$0 \le k, |s| \le 10^5$。

题解

这个题有非常多的方法，由于 $lose$ 这个串很普通，没有 $boarder$，所以办法很简单，我们尽量把匹配成功数量多的位置凑成 $lose$ 删掉即可。

具体实现的时候可以考虑按 2^{16} 种情况，按 1 的个数从多到少分别去暴力一遍，或者直接只暴力一遍，判断每个位置属于哪种情况，然后丢入 $vector$ 中，一次性贪心。

容易 wa 的地方是， 会忘记还能凭空拿 4 个字符造 $lose$。

染色（定位：简单题）

给一个长度为 n 的序列，每次操作可以花费 $c_i + (r - l + 1)$ 的代价，把区间 $[l, r]$ 染成染色 i。

问：最少多少钱才能把整个序列染成颜色 $1, 2, 3, \ldots, n$。

$n \le 2 \times 10^5, 1 \le a_i, c_i \le n$。

题解

复杂度计算题。

考虑算每一种颜色的时候，就是给你一个 01 序列做上述事情，那么我们先把连续的 0 和连续的 1 缩起来，这样对于所有颜色，连续的 0 和 1 的段一共只有 $O(n)$ 个。

对于每一种颜色，做法就是直接贪心了，我们先假设所有连续的 0 段都用了一次染色的机会，然后再考虑合并两次染色是否会省代价，显然就是在比较他们中间的 1 的长度以及 c_i。

最大公因数（定位：简单题）

给定两个正整数 a, b，输出满足条件的 x 满足 $1 \le x \le 10^{18}$，$\gcd(a, x) = 1, \gcd(b, x) > 1$。

$T \le 10^4, a, b \le 10^{18}$。

题解

这个题输出 b 的倍数看起来就没啥意义，所以考虑输出的数字应该是 b 的约数，那么可以考虑让 x 一开始是 b。

但是 x 里面不能含有 a，所以考虑不停给 x 除以 $\gcd(a, x)$ 直到无解。

这样的过程最多 $\log_2 b$轮，能接受。

学生（定位：简单题）

一个长度为 n 的序列，第 i 位是 a_i。

对于一个数字 a_i，如果左侧中有大于等于它的，右侧有小于等于它的，或者反过来成立，则可以删掉它。

问最终序列最少还剩几个数字。

$n \le 2 \times 10^5$。

题解

考虑全局最大值，至少得剩一个删不掉，全局最小值，最少得剩一个删不掉；a_1, a_n 一定删不掉。

所以考虑能否删完其他位置。

假设 mx, mn 都不是 a_1, a_n，那么对于一个数字，如果在mx, mn 中间，则直接删了，如果在他俩左边或者右边，则可以选他们中的一个，搭上 a_1/a_n 来删。

所以最后的答案是 $2+$ [全局严格最大值在中间] $+$ [全局严格最小值在中间] 。

打字（定位：简单题~中档题）

给定一个数组 $cost_{x,y}$ 表示从字符 x 切换到字符 y 的代价，t 表示把光标左右移动一格，或者是敲一下键盘的代价。

问打印出目标串的最小总代价是多少。

$|s| \le 20$。

题解

一个明显的状压dp，我们考虑记录当前哪些位置被打印了出来，光标在哪里，就可以暴力枚举下一个字符打在哪里来转移了。

$dp_{sta,i} -> dp_{sta|2^j,j}$，代价是 $cost_{s_i,s_j} + t + t \times \mathrm{num}_{i,j}$。其中 $num_{i,j}$ 表示 sta 这个二进制数从 i 到 j 过程中 1 的个数。

复杂度 $O(2^n \times n \times n)$，实际上由于 sta 和 i 相互的限制，只有一半的常数，但因为好多验题人写的非常慢（std最慢的点不超过 $800\mathrm{ms}$，其他人两倍以上时间，所以时限开的大。

买股票（定位：中档题）

有一个数字 v ，以及 n 个操作，每个操作分别是 $+a_i$ 或者是 $\times a_i$ 的形式，让你重排这 n 个操作的顺序，使得算完前 $1, 2, 3, \ldots, n$ 个操作时候，变量 v 的均值最大。

$n \leq 30$。

题解

打表或者大胆猜结论或者是数学水平拉满，很容易看出一个结论，无论最终顺序怎么排，必须满足，$+$ 操作对应的值从大到小，$\times$ 操作对应的值从大到小。（这个结论严谨的证明过了，但是由于太臭太长了就没有放出来）

那么，搜索的时候每次就只有两种选择了，复杂度最坏是 $C_n^{n/2}$。

另一种超快的办法是，考虑采用bfs，在搜索的每一层维护 $\{now, sum\}$ 对，对于每一层的所有 $\{now, sum\}$对，显然只有最外层的阶梯状的一些点是有用的，这些点在随机数据下非常少，也想不到有什么特别可以卡的方式。

以下是本题结论正确性证明的部分，不想看的请跳过：

最大化题面当中的 $\frac{\sum_{i=1}^{n} v_i}{n}$，显然与最大化 $\sum_{i=0}^{n} v_i$ 等价，之后将由 ans 代替该式，不多赘述。

为了方便证明，我们先做出一些约定。

假设我们最终排出的操作序列为 (op_i, x_i)，其中 op_i 为操作类型（乘或加），x_i 为该操作的对应参数。特别的，我们强制认为 $op_0 = +, x_0 = v$。

由此定义一个基于 (op_i, x_i) 的序列 $\{d_i\}$，其递推式为 $d_i = \begin{cases} 1 & i \le 0 \\ d_{i-1} & i > 0 \ \wedge \ op_i = + \\ d_{i-1}x_i & i > 0 \ \wedge \ op_i = \times \end{cases}$。不难考虑 d_i 的实际意义为所有乘操作的前缀积。

再在次基础上，定义一个序列 $\{s_i\}$，其递推式为 $s_i = \begin{cases} 0 & i < 0 \\ s_{i-1} + d_i & i \ge 0 \end{cases}$。定义权值序列 $u_i = \frac{s_n - s_{i-1}}{d_{i-1}}$，此时我们不难发现 $ans = \sum_{i=0}^{n} [op_i = +] x_i u_i$，即我们考虑将所有加操作对 ans 的贡献分开计算，u_i 即为第 i 个操作对 ans 的贡献系数。由 u_i 的实际意义也可知若我们任意调整 x_{i-1} 的值甚至任意调整 x_k 的值（$k < i$），u_i 的值并不会发生变化。**这很重要。**

接下来，我们开始证明我们的结论。

引理. $\forall 0 \le i < n, u_i > u_{i+1}$

比较显然，注意到对于所有 $i \ge 1$ 有 $x_i > 0$ 以及 $d_0 = 1 > 0$，故所有 $d_i > 0$，故所有 $s_i > s_{i-1}$，即 s_i 为递增序列；再由于 $x_i > 1$，我们有 $d_i \ge d_{i-1}$，等号取到当且仅当 $op_i = +$。于是 $u_i = \frac{s_n - s_{i-1}}{d_{i-1}} > \frac{s_n - s_i}{d_i} = u_{i+1}$。根据 u_i 的实际意义理解不难。

引理推论 $\{u_i\}$ 为递降序列。

结论第一部分. 在 op_i 完全相同且确定的所有排列方案中，存在一个最优方案有性质 $\forall 1 \le i < j \le n$ 且满足 $op_i = op_j = +$，有 $x_i \ge x_j$。

显然，根据引理我们有 $\forall 1 \le i < j \le n, u_i > u_j$，若最优方案中存在一对满足上述条件的 i 与 j，且 $x_i < x_j$ 的，我们不难发现交换 x_i 与 x_j 后，ans 增大了 $(u_i - u_j)(x_j - x_i)$（显然为正），与此方案为最优方案矛盾。

结论第二部分. 在 op_i 完全相同且确定的所有排列方案中，最优方案要么均比全局最优方案劣，要么存在一个最优方案有性质 $\forall 1 \le i < j \le n$ 且满足 $op_i = op_j = \times$，有 $x_i \ge x_j$。

感性理解：如果将大的乘法放在后面更优的话，我们为什么不考虑把前面那个小的乘法直接**整个移动到**它后面呢？

下为严谨证明。

假设存在某种 op_i 的排列，在 op_i 固定的所有方案中，所有最优方案均存在一对满足上述条件的 i 和 j，且 $x_i < x_j$。

不难想到一定存在一组“相邻”的 i 和 j 满足类似限制，即一定存在 i, j 满足 $i < j, op_i = op_j = \times, x_i < x_j, \forall i < k < j, op_k = +$。取满足此限制的任意一组，我们对这组 i 和 j 进行**交换**考察。

按照题设，我们将序列 $\{v_i\}$ 生成出来。考虑进行 ans 贡献的拆分与组合，不难注意到 ans 形如 $u_{i-1}v_{i-1} + \sum\limits_{k=i+1}^{j-1} x_k u_k + E$，其中 E 为一个常数。而我们再次发现 $\forall i < k < j, u_k = u_j + j - k = u_{j+1}x_j + x_j + j - k$（利用 u_i 定义式不难推出），且 $u_{i-1} = \frac{s_n - s_{i-2}}{d_{i-2}} = \frac{s_n - s_{i-1}}{d_{i-1}} + 1 = \frac{s_n - s_i + d_i}{d_i} \times \frac{d_i}{d_{i-1}} + 1 = u_{i+1}x_i + 1 = u_{j+1}x_j x_i + x_i x_j + (j - i - 1)x_i + 1$，我们注意到这里面的 $u_{j+1}, x_i x_j, i, j, 1$ 均为常数，故我们又可以将 ans 写作 $Ax_i + Bx_j + C$，其中 A, B, C 均为常数，并且经过化式子我们得知 $A = v_{i-1}(j - i - 1), B = (u_{j+1} + 1)\sum\limits_{k=i+1}^{j-1} x_k, C = E + v_{i-1}(u_{j+1} + 1)x_i x_j + v_{i-1} + \sum\limits_{k=i+1}^{j-1} x_k(j - k)$。

由于此方案为最优方案，故我们可以得知由于 $x_i < x_j$，故 $A < B$，否则交换 x_i, x_j 会使 ans 增加一个非负数。这看上去是可能的。

然而，事实上，我们考察另一种情况：将操作 (op_i, x_i) 提出来，重新放置在操作 (op_j, x_j) 的后面。也就是说我们由此定义出了一个新的操作序列 (op'_i, x'_i)，由此我们也可以生成 d'_i, s'_i, u'_i, v'_i。

（下文的 i 与 j 发挥下标作用而非指代操作，以及注意下文的含撇与不含撇的序列名称，不要搞混了）

注意到由于操作本身并没有变化太多，我们有 $v'_{i-1} = v_{i-1}$，$u'_{j+1} = u_{j+1}$。

此时我们的 ans' 可以看作 $u'_{i-1}v_{i-1} + \sum\limits_{k=i}^{j-2} x'_k u'_k + E = u'_{i-1}v_{i-1} + \sum\limits_{k=i+1}^{j-1} x_k u'_{k-1} + E$，这里的 E 与前文的 E 是相同的，不难通过实际意义理解。

类似的，我们有 $\forall i - 1 \le k \le j - 2, u'_k = u'_{j-1} + j - k - 1 = (u'_j + 1)x'_{j-1} + j - k - 1 = ((u_{j+1} + 1)x'_j + 1)x'_{j-1} + j - k - 1 = u_{j+1}x_i x_j + x_i x_j + x_j + j - k - 1$。

故化式子知 ans 为 $v_{i-1}x_j + x_j \sum\limits_{k=i+1}^{j-1} x_k + v_{i-1}(u_{j+1} + 1)x_i x_j + v_{i-1}(j - i - 1) + v_{i-1} + (u_{j+1} + 1)x_i x_j \sum\limits_{k=i+1}^{j-1} x_k + \sum\limits_{k=i+1}^{j-1} x_k(j - k) + E$，进一步化为 $(v_{i-1} + \sum\limits_{k=i+1}^{j-1} x_k)x_j + A + Bx_i x_j + C$。

由于 $A < B$ 且 $1 < x_i < x_j$，我们有 $A + Bx_ix_j = A + Bx_j + Bx_j(x_i - 1) \geq A + Bx_j + A(x_i - 1) = Ax_i + Bx_j$，外加上 $(v_{i-1} + \sum_{k=i+1}^{j-1} x_k)x_j > 0$，我们得知 $ans' > ans$，也就是说，**我们成功证明了将操作 (op_i, x_i) 提出来，重新放置在操作 (op_j, x_j) 的后面，会使得方案严格变优。**

故结论第二部分证明完毕。

又由于全局最优解一定存在，且其不劣于全局最优解，故其一定满足所谓加操作递降、乘操作递降，形式化版本参考两条分步证明的结论。

至此，我们完全证明了本题最为关键的结论。

LOCK S（定位：中档题）

一棵树，每个点有权值 v_i，两个人玩游戏，轮流决策：

第一个人的策略是：随便占领一个未被占领的节点。

第二个人的策略是：向下走到未被占领的，权值最大的节点。

不能操作时游戏结束。

问：游戏结束时，两个人占领节点的权值差最大是多少。

$n \le 2 \times 10^5, v_i \le 10^9$。

题解

有的验题的队伍在想dp，不懂为什么会觉得这是一个dp。

考虑游戏要在哪里结束，那么就能确定这样三个信息：

（1）哪些点是先手必须要占领的，哪些点是后手占领过的。

（2）先手除了这些必须占领的点，还能额外占领几个点。

（3）哪些点是能够被先手自由占领的。

如果我们能对第（3）种信息，维护一个权值线段树，那么我们直接在权值线段树上二分找最大的一些点即可。

实际上这个信息很容易被维护，我们只需要建立一棵权值线段树，先把所有点都加入线段树中，然后从根节点开始dfs，暴力枚举后手下一步会走到哪个点，那么（1）中的点就确定好了，我们在权值线段树中把这些点的贡献减掉，然后dfs去这个子树。

这个点的答案在dfs进入的这个点的时候，在权值线段树上二分。

当我们回溯回来的时候，把刚才从权值线段树中减去的东西再加回来即可。

时间复杂度 $O(n\log_2 n)$。

魔法（定位：中档题~难题）

给一个序列 $b_1, \ldots, b_n$，以及一个参数 k，每次可以找一个位置 $k \le x \le n-k$，另 $t = b_{x+1} - b_x$，然后把$1 \le d \le k$ 内的所有 b_{x+d} 减去 t，b_{x-d+1} 加上 t 。

问最终能得到多少种不同的序列。

$n \le 5 \times 10^5, 1 \le k \le \frac{n}{2}, |b_i| \le 10^9$。

题解

这是一个脑洞题，本来的idea是，给定某种操作，实际上是交换了一个数组差分数组的第 x 和 $x+d$ 项，问有多少种方案。

这样实际上就是我们找一下连通块，连通块内可以随便换，那就相当于是直接组合数一下就把方案数算出来了。

后来包装了一下这个题意，就变成了上述这个看起来比较复杂的题意了。

我们首先构造一个长度为 $n+1$ 的数组 a，满足 $a_{i>n+1}=0, a_i=b_i+a_{i+k}$。

然后对 a 数组求差分 $d_i=d_i-d_{i-1}$。

然后就变成上面的题意了。

对于数学直觉好的选手应该可以一眼秒掉。

关灯（定位：难题）

n 盏灯，一共 m 个开关，第 i 个开关管着 sta_i 的二进制状态，会给这些灯取反。

现在给定灯的初始状态，问最少选多少代价的开关才能把灯关掉。

一个特殊的限制是：对于第 i 个开关，满足这个开关管着的灯里面，至少有一盏灯，被不超过一个前 $i-1$ 个开关管着，但是这个顺序被打乱了。

$n, m \le 60, c_i \le 10^5, m \le 2n$。

题解

看起来这个题很像线性基，但实际上没啥关系。

我们考虑那个特殊限制有什么作用，看起来是，我们按照某种方式拓扑排序，总能找到一个灯，这个灯此时最多只被两个开关管着，这时我们可以把关系到的这两个开关从图中删掉。

复述一遍上面的结论：我们把开关和灯进行了一些关联， 使得任意时刻，一个灯最多关联两个开关。

那么我们可以按照灯的顺序进行dfs，对于每一个灯，假设跟他相关的开关有 k 个 $(k \le 2)$，那么只有 2^{k-1}种情况可以把它关掉。

这样搜索的话，时间复杂度就是 $O(2^{\frac{m}{2}})$ 了，过不了。

于是我们考虑进行一些折半操作。

我们把这些开关均等的分成三份，对于第三份，暴力 $O(2^{\frac{m}{3}})$ 去dfs所有情况，然后把它的影响存进map里面。

然后去dfs前 $\frac{2}{3}m$ 个开关影响的灯，最后 $\frac{m}{3}$ 的部分在map里面查即可。

平衡下来时间复杂度刚好是 $O(2^{\frac{m}{3}} \log 2^{\frac{m}{3}})$。

噩梦（定位：难题）

一个长度为 n 的序列，第 i 个位置权重 w_i，颜色是黑白之一。

每一秒，每个点会检查自己所在同色连通块的权重，如果其左右之一存在权重比自己大的异色连通块，则自己会变色，所有变色是同步进行的。

有 q 个询问，每个问题是问第 t 秒第 x 个点是什么颜色。

$n \le 2 \times 10^5, w_i \le 10^9, t_i \le 10^9$。

题解

考虑一个简单情况，假设每个连通块从左到右刚好是递降的，比如 $[5,4,3,2,1]$，会怎么样？

那么，这个序列每一秒，会被“吃掉”一位，在后续每一秒变成：

$[9,3,2,1],[12,2,1],[14,1],[15]$。

由此可以想到，**局部最大值总会在一秒内吃掉其左右的两个点，而其他点的颜色会根据其距离局部最大值来交替变换。**

那么，我们可以考虑按秒来模拟，用双向链表把所有当前存在的点串起来，用一个数组或者set之类的东西记录当前的局部最大值。对于一次查询，我们需要知道当前这个点，距离局部最大值的距离是多少，所以我们还需要一个树状数组，来维护当前哪些点是局部最大值。

时间复杂度 $O(n\log n)$。

这个题很容易写挂，比如某一秒一个局部最大值吃掉了另一个局部最大值，或者是某个局部最大值退出了“局部最大值”序列，或者是比如 $[1,0,1,0,1]$，如果这三个 1 是局部最大值，那么下一秒这五个位置会合成一个大连通块。

吃糖（定位：很难的题）

一个变量 v，初始等于 0，共 n 天，给了一个长度为 n 的01序列，第 i 天，如果 $s_i = 1$，那么一半的概率让变量 $v + 1$，一半概率不变，如果 $s_i = 0$，一半的概率让变量 $v - 1$，一半的概率不变。

当这个事情结束以后，问变量 v 在整个过程中**最大值**的期望。

$n \le 5 \times 10^5$。

暴力

为方便表述，不妨假设将所有 1 的位置替换成数字 1，所有 0 的位置替换为数字 -1。

那么题目给出的是一个只包含 1 和 -1 的序列 a，所求的答案为序列 a 所有子序列的最大前缀和之和。

设 $dp_{i,j,k}$ 为从前 i 个位置中选了若干个位置组成一个子序列，满足这个子序列的最大前缀和 j，且整个串当前的和为 k，方案数是多少。

边界状态为 $dp_{0,0,0}=1$。

考虑 $dp_{i-1,j,k}$ 能往哪些状态转移：

1. 不选 i 这个位置，有转移 $dp_{i-1,j,k}\to dp_{i,j,k}$；
2. 选了 i 这个位置，再按照 a_i 进行分类讨论：
    1. $a_i=-1$，有转移 $dp_{i-1,j,k}\to dp_{i,j,k-1}$；
    2. $a_i=1$，有转移 $dp_{i-1,j,k}\to dp_{i,\max(j,k+1),k+1}$。

答案就是 $\sum\limits_{j,k} dp_{n,j,k}\times j$。

时间复杂度为 $O(n^3)$。

暴力优化

不难发现，求一个序列的最大前缀和，相当于先给这个序列做一个前缀和，然后求这个序列的**严格**前缀最大值个数。

考虑拆贡献，枚举每一个位置 i，统计有多少个子序列包含 i 这个位置，并且 i 这个位置的前缀和**严格**大于前面所有位置的前缀和。

不难发现，i 的前缀和是严格前缀最大值，相当于 $a_i = 1$，并且 $i-1$ 的前缀和是（可以不严格的）前缀最大值。

设 $dp_{i,j}$ 表示从前 i 个位置中选了若干位置组成一个子序列，对于选出的序列，设其最大前缀和为 x，整个序列的和为 y，满足 $x-y=j$ 的方案数。

不难发现，若 $j=0$，则表示整个序列的和与最大前缀和的值相等，此时只需要在序列末尾添加一个 1，整个序列的权值就会增加 1。

初始状态为 $dp_{0,0}=1$，考虑 $dp_{i-1,j}$ 能往哪些位置转移。

1. 没有选 i 这个位置，有转移 $dp_{i-1,j} \to dp_{i,j}$；
2. 选了 i 这个位置，按照 a_i 进行分类讨论：
    1. $a_i = 1$，则序列的和会增加 1，整个序列的权值有可能增加 1，有转移 $dp_{i-1,j} \to dp_{i,\max(0,j-1)}$；
    2. $a_i = -1$，则序列的和会减少 1，整个序列的权值一定不变，有转移 $dp_{i-1,j} \to dp_{i,j+1}$。

答案即为 $\sum\limits_{a_i=1} dp_{i-1,0} \times 2^{n-i}$。

时间复杂度为 $O(n^2)$。

正解

考虑对上面这个 $O(n^2)$ 的 dp 进行优化。

可以观察到一些性质，对于所有 $j \geq 1$ 的 $dp_{i,j}$，他们的转移都是比较“整齐”的。

什么叫整齐呢，具体一点，对于一个 i，要么所有的 j 都同时往 $j, j-1$ 这两个位置转移，要么所有的 j 都同时往 $j, j+1$ 这两个位置转移。

这一部分的转移相当于是整体卷上了一个 $x+1$。

只有 $j=0$ 这一个单点他的转移比较奇特，可能会出现自己往自己转移两次的情况。

同时还有一个关键的地方，那就是统计答案的时候我们其实只用到了每一个 $dp_{i,0}$ 的值。

再来思考，假设我们知道所有 dp_{l-1} 的值，能否比较快速地得到 dp_r 的值。

不难发现，对于所有 $j \geq r-l+1$ 的 $dp_{l-1,j}$，它的转移是可以比较快的处理的，因为其转移的每一步都不会触碰到某一个 $dp_{k,0}$。

所以这一部分可以直接卷上一个 $\sum_{i=0}^{r-l+1} \binom{r-l+1}{i} x^i$ 转移到 dp_r 去。

对于 $j \leq r-l+1$ 的部分，直接暴力做 dp 即可，顺便还可以完成答案的统计。

这样可以在 $O((r-l+1)^2 + n\log n)$ 的复杂度内完成 $[l, r]$ 这一段 dp 值的转移。

那么我们直接 B 个位置分一块，每一块都做上述的转移，就可以在 $O(\frac{n^2}{B}\log n + nB)$ 的复杂度内完成这道题。

取 $B = \sqrt{n\log n}$ 可以得到复杂度为 $O(n\sqrt{n\log n})$ 的做法，大概过不了。

继续利用上面的思路进行优化，但是将分块换成分治。

考虑这么一个问题，对于一个区间 $[l, r]$，我们已知 $dp_{l-1,j}$ 的值，其中第二维的范围为 $0 \leq j < r-l+1$。

现在需要比较快地求出 $dp_{r,j}$ 的值。

考虑利用分治的思想，若 $l = r$，则 dp_{l-1} 最多只有一项，暴力转移即可。

若 $l < r$，设区间中点为 $m = \left\lfloor \frac{l+r}{2} \right\rfloor$，不妨先根据 $dp_{l-1,j}$ 的值求出 $dp_{m,j}$ 的值，再求出 $dp_{r,j}$ 的值。

对于前半部分，考虑 $j \geq m - l + 1$ 的部分，其在转移的时候不会碰到 $dp_{k,0}$，所以可以直接像上面说的一样，用 NTT 卷过去。

对于 $j < r - l + 1$ 的部分，直接递归调用 $[l, m]$ 解决。

这样就成功求出了所有 $dp_{m,j}$ 的值，做一遍几乎一样的事情转移到 $dp_{r,j}$ 即可。

时间复杂度为 $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$，可以通过本题。